

March 26, 2026

Pinocchio Token

Smart Contract Patch Review

Placeholder text for the Smart Contract Patch Review content.

Contents

About Zellic	3
---------------------	----------

1. Introduction	3
------------------------	----------

1.1. Scope	4
------------	---

1.2. Disclaimer	6
-----------------	---

2. Patch Review	6
------------------------	----------

2.1. Out-of-scope suggestion	7
------------------------------	---

3. Patch Review	7
------------------------	----------

3.1. PR 113 - Update amount on self-unwrap lamports	8
---	---

3.2. PR 120 - Add delegate support to unwrap lamports	8
---	---

3.3. PR 121 - Add self-revoke delegate support	9
--	---

3.4. PR 132 - Sync native updates rent exempt reserve	9
---	---

3.5. PR 133 - Allow for token-2022 multisigs to be used	9
---	---

3.6. PR 137 - Add optional rent sysvar account	9
--	---

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Introduction

During the period of March 18th to March 20th, 2026, we were asked to review a minor patch to Pinocchio Token, which fixed a minor out-of-bounds issue that did not have any security impact.

1.1. Scope

The engagement involved a review of the following targets:

Pinocchio Token Programs

Type	Rust
Platform	Solana
Target	token
Repository	https://github.com/solana-program/token
Version	90b8f5c8b4b6644bbed3c905450ffbdb20c83752
Programs	p-token
Target	token
Repository	https://github.com/solana-program/token
Version	9ba3e556006b08e47f1a5dea678fa839fda4a23c
Programs	p-token

Target	token
Repository	https://github.com/solana-program/token ↗
Version	bf5d08f9a33275c515edf4d043ff07fa4850d9a4
Programs	p - token
Target	token
Repository	https://github.com/solana-program/token ↗
Version	4a97dba2fddec32c6b08c7c047c134fb1d266958
Programs	p - token
Target	token
Repository	https://github.com/solana-program/token ↗
Version	f403c97ed4522469c2e320b8b4a2941f24c40a5e
Programs	p - token
Target	token
Repository	https://github.com/solana-program/token ↗
Version	065786e22bc7dfdb74515abea7995bb9eb9ad7f0
Programs	p - token

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Nathanial Lattimer
✈ Engineer
d0nut@zellic.io ↗

Li Shi
✈ Engineer
li@zellic.io ↗

1.2. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.

2. Patch Review

We performed an extensive review of each of the six PRs that were in scope for this engagement. During the assessment, we looked for violations of memory safety, expected behavior, and undefined behavior. Through manual review and some automated approaches, we discovered no findings in the PRs.

2.1. Out-of-scope suggestion

We suggest one improvement to reduce risk, which is in code outside of the scope of the audit. The `load_unchecked` and `load_mut_unchecked` functions restrict the types that can be loaded to those that implement the trait `Transmutable`. We suggest that `Transmutable` be defined as a sealed trait and to separately include in-line `const` assertions to ensure alignment requirements are upheld.

This issue has been acknowledged by Anza, and a fix was implemented in commit [ac14e9f0](#).

3. Patch Review

This section documents notable and minor changes applied to the in-scope PRs.

3.1. PR 113 - Update amount on self-unwrap lamports

This PR changed the behavior of the unwrap instruction to change the amount field on the native token account in self-unwrap conditions. This allows a partial unwrapping of the native token account.

3.2. PR 120 - Add delegate support to unwrap lamports

This PR adds support for native token account unwrapping by delegates. This is implemented by checking if the provided authority for the transaction is the delegate. If so, a delegate-specific authorization flow is entered. Otherwise, the traditional ownership check is performed.

```
// Validates the authority (delegate or owner).

if source_account.delegate() == Some(authority_info.key()) {
    // SAFETY: `authority_info` is not currently borrowed; in the case
    // `authority_info` is the same as `source_account_info`, then it cannot be
    // a multisig.
    unsafe { validate_owner(authority_info.key(), authority_info, remaining)?
    };

    let delegated_amount = source_account
        .delegated_amount()
        .checked_sub(amount)
        .ok_or(TokenError::InsufficientFunds)?;

    source_account.set_delegated_amount(delegated_amount);

    if delegated_amount == 0 {
        source_account.clear_delegate();
    }
} else {
    // SAFETY: `authority_info` is not currently borrowed; in the case
    // `authority_info` is the same as `source_account_info`, then it cannot be
    // a multisig.
    unsafe { validate_owner(&source_account.owner, authority_info, remaining)?
    };
}
```

3.3. PR 121 - Add self-revoke delegate support

This PR adds support for delegates to revoke themselves. It was largely a simple change which inspected if the provided authority was the listed account delegate.

3.4. PR 132 - Sync native updates rent exempt reserve

This PR added support to update the withheld rent on native token accounts. This was done to support a long-discussed change to Solana's rent. This change opted to access the Rent sysvar via syscall rather than relying on the Rent account being provided, avoiding a change to the public interface.

3.5. PR 133 - Allow for token-2022 multisigs to be used

This change allowed Token-2022 multisigs to be used, whereas previously multisigs were required to belong to the original Token program.

```
if unlikely(  
    owner_account_info.data_len() == Multisig::LEN  
    && (owner_account_info.is_owned_by(&TOKEN_PROGRAM_ID)  
        || owner_account_info.is_owned_by(&spl_token_2022::ID)),  
) {  
    ...  
}
```

3.6. PR 137 - Add optional rent sysvar account

This change augments PR - 132 by allowing the caller to optionally provide the Rent sysvar account in the native sync instruction. This had the effect of reducing the compute cost of the call considerably, despite the added complexity.

```
/// Creates a `SyncNative` instruction with the Rent sysvar account  
/// added to the accounts list.  
pub fn sync_native_with_rent_sysvar(  
    token_program_id: &Pubkey,  
    account_pubkey: &Pubkey,  
) -> Result<Instruction, ProgramError> {  
    let mut instruction = sync_native(token_program_id, account_pubkey)?;  
    instruction  
        .accounts  
        .push(AccountMeta::new_readonly(sysvar::rent::id(), false));
```

```
Ok(instruction)
}
```

The following is the change in the `process_sync_native` function which optionally uses a provided `Rent` sysvar, or accesses it via `syscall`.

```
let rent_exempt_reserve = if let Some(rent_sysvar_info) = remaining.first() {
    // SAFETY: single immutable borrow to `rent_sysvar_info`; account ID and
    // length
    // are checked by `from_account_info_unchecked`.
    let rent = unsafe { Rent::from_account_info_unchecked(rent_sysvar_info)? };
    rent.minimum_balance(native_account_info.data_len())
} else {
    Rent::get()?.minimum_balance(native_account_info.data_len())
};
```

Despite the naming, `Rent::from_account_info_unchecked` does validate that the provided account is the `Rent` sysvar account. The `unchecked` suffix refers to a runtime borrowing check that is not performed here.